SmartAgri AI Platform — Comprehensive Project Report

> **Project Name:** SmartAgri — AI-Powered Precision Agriculture Advisory Platform **Stack:** 100% Pure MERN (MongoDB Atlas · Express.js · React 19 · Node.js) **Version:** 1.0.0 | **Date:** October 2026

1. Project Concept & Objective

SmartAgri is a full-stack, AI-driven digital farming advisor built to help Indian farmers make smart, data-driven decisions in real time. The platform bridges the gap between cutting-edge technology (IoT sensors, AI, Cloud databases) and the ground-level needs of farmers — available in **four languages**: English, हिन्दी (Hindi), தமிழ் (Tamil), and తెలుగు (Telugu).

1.1 Problem Statement

Indian smallholder farmers (< 5 acres) lack affordable access to:

- Real-time soil and climate data for their specific micro-location.
- Expert-level agronomic advice in their native language.
- Timely information on government schemes and market prices.
- Disease diagnosis without travelling to an agriculture office.

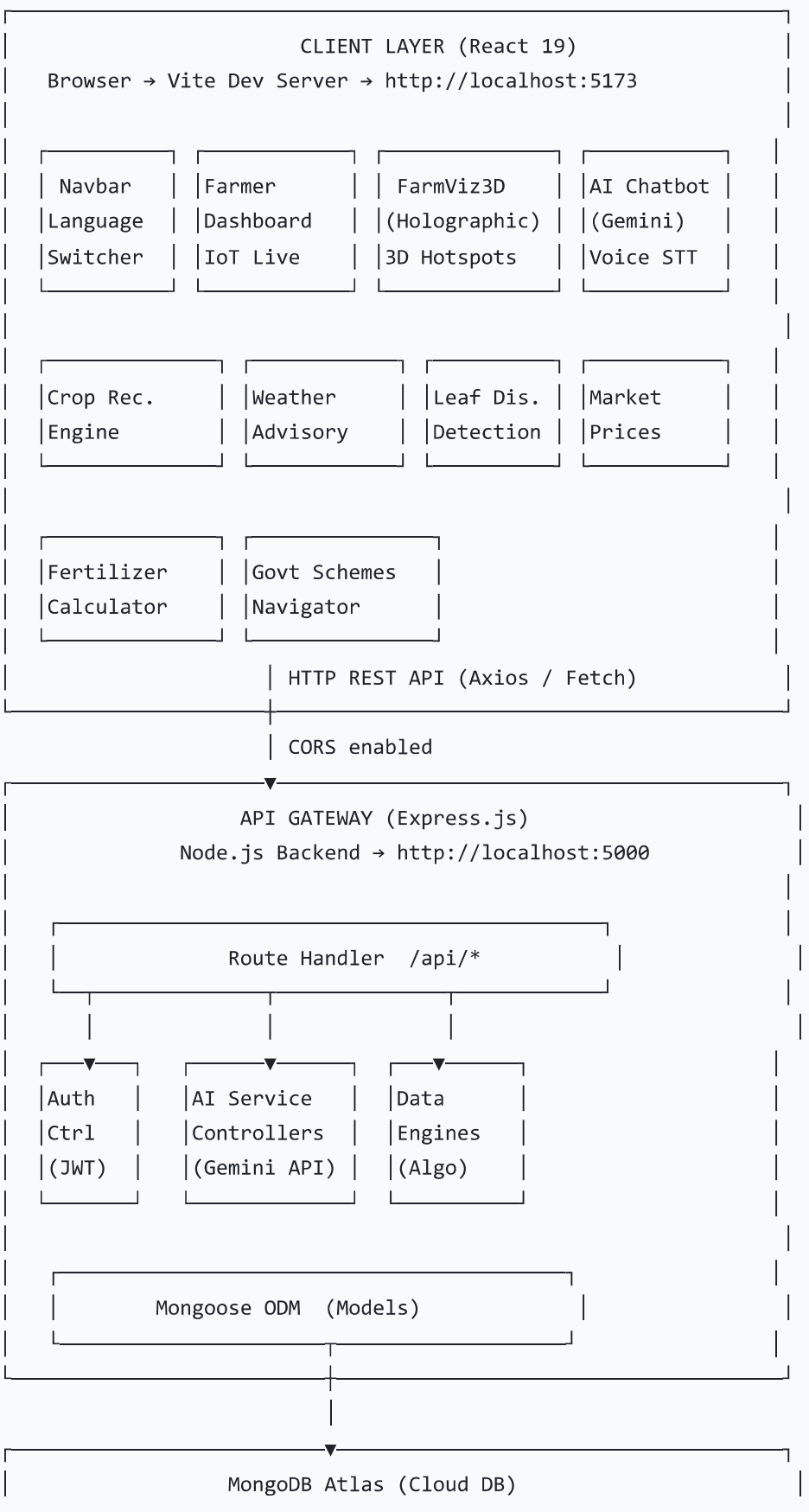
1.2 Solution

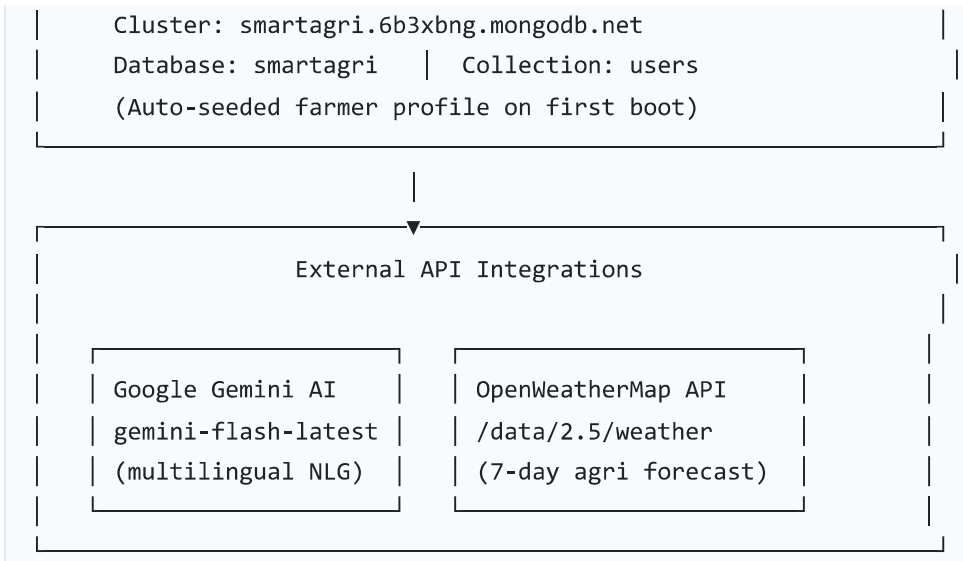
SmartAgri delivers a **web-based advisory dashboard** that integrates:

- Live IoT soil telemetry simulation.
- Google Gemini AI-powered multilingual chatbot.
- Rule-based crop, fertilizer, and irrigation recommendation engines.
- OpenWeatherMap-driven weather advisory.
- Leaf disease detection via image analysis.
- Real-time APMC market price tracker.
- Government scheme navigator.

- Interactive 3D farm visualizer with holographic hotspots.

2. System Architecture





3. Data Flow & Request Lifecycle

4. API Endpoints Reference

Method	Endpoint	Controller	Purpose
`POST`	`/api/auth/register`	authController	Register new farmer account
`POST`	`/api/auth/login`	authController	Login & issue JWT
`GET`	`/api/auth/profile`	authController	Get authenticated profile (JWT required)
`POST`	`/api/crop/recommend`	cropController	AI crop recommendation by NPK + soil
`GET`	`/api/weather/advisory`	weatherController	Live weather + 7-day agri forecast
`POST`	`/api/disease/detect`	diseaseController	Leaf disease image classification
`POST`	`/api/irrigation/calculate`	irrigationController	Smart drip/flood irrigation plan
`POST`	`/api/fertilizer/recommend`	fertilizerController	Balanced NPK fertilizer schedule
`GET`	`/api/market/prices`	marketController	APMC mandi crop price tracker
`POST`	`/api/market/predict-profit`	marketController	Seasonal profit forecast
`POST`	`/api/chat/ask`	chatbotController	**Gemini AI** multilingual chatbot
`GET`	`/api/schemes`	schemeController	Government agriculture schemes list
`GET`	`/api/iot/telemetry`	iotController	Real-time simulated IoT soil sensor data
`POST`	`/api/iot/pump`	iotController	Toggle smart water pump ON/OFF
`GET`	`/health`	server.js	System health + MongoDB status check

5. Full File Structure

```
d:\Smartagri\
│
├── package.json                          ← Root workspace config
├── README.md
│
├── server\                               ← Node.js + Express Backend
│   ├── server.js                         ← Entry point, MongoDB connect, app bootstrap
│   ├── package.json                      ← Backend dependencies (express, mongoose,
bcryptjs…)
│   ├── .env                              ← Secrets: MONGODB_URI, GEMINI_API_KEY,
WEATHER_API_KEY
│   │
│   ├── routes\
│   │   └── api.js                        ← All REST route declarations
│   │
│   ├── controllers\                      ← Business logic layer
│   │   ├── authController.js             ← Register, Login, JWT profile
│   │   ├── chatbotController.js          ← Gemini AI + fallback agronomy engine
│   │   ├── cropController.js             ← NPK scoring algorithm (9 crops)
│   │   ├── diseaseController.js          ← Leaf disease image classification
│   │   ├── fertilizerController.js       ← NPK fertilizer calculator
│   │   ├── iotController.js              ← IoT sensor simulation + pump toggle
│   │   ├── irrigationController.js       ← Smart irrigation scheduler
│   │   ├── marketController.js           ← APMC market prices + profit predictor
│   │   ├── schemeController.js           ← Government scheme navigator
│   │   └── weatherController.js          ← OpenWeatherMap + agri-advisory engine
│   │
│   ├── models\
│   │   └── User.js                       ← Mongoose schema: farmer profile
│   │
│   └── middleware\
│       └── auth.js                       ← JWT authentication middleware
│
└── client\                               ← React 19 + Vite Frontend
    ├── index.html                        ← SPA root HTML
    ├── vite.config.js                    ← Vite bundler config (proxy → :5000)
    ├── tailwind.config.js                ← Tailwind CSS theme (cyber neon + nature)
    ├── postcss.config.js
    ├── package.json                      ← Frontend deps (React, Framer Motion, Lucide)
    │
    ├── public\
    │   ├── favicon.svg
    │   ├── realistic_smart_farm.jpg      ← Hero banner image (day mode)
```

```
    │   └── smart_farm_thermal.jpg      ← Hero banner image (thermal / night mode)
    │
    └── src\
        ├── main.jsx                    ← React DOM root render
        ├── App.jsx                     ← Root component: state, routing, TTS readout
        ├── App.css                     ← Global CSS overrides
        ├── index.css                   ← Tailwind base + custom keyframes
        │
        ├── locales\
        │   └── translations.js         ← i18n dictionary: en / hi / ta / te
        │
        ├── services\
        │   └── api.js                  ← Axios/Fetch service layer → /api/*
        │
        ├── assets\
        │   ├── hero.png
        │   └── react.svg
        │
        └── components\
            ├── Navbar.jsx              ← Navigation bar, language switcher, theme toggle
            ├── AuthModal.jsx           ← Login / Register modal (JWT)
            ├── FarmerDashboard.jsx     ← Main IoT dashboard + audio briefing
            ├── FarmVisualizer3D.jsx    ← Cyber 3D holographic farm map
            ├── AiChatbot.jsx           ← Voice + text Gemini AI chatbot
            ├── CropRecommendation.jsx  ← NPK-based crop advisor form
            ├── WeatherAdvisory.jsx     ← Weather + 7-day agri forecast
            ├── LeafDiseaseDetection.jsx← Drag-and-drop leaf image analyser
            ├── SmartIrrigation.jsx     ← Drip/flood irrigation planner
            ├── FertilizerCalculator.jsx← NPK fertilizer dose calculator
            ├── MarketPriceProfit.jsx   ← Mandi prices + profit predictor
            └── GovtSchemes.jsx         ← PM-KISAN, KUSUM, PMFBY scheme browser
```

6. Core Modules Explained

6.1 Authentication System (`authController.js` + `auth.js`)

- **Technology:** `bcryptjs` for password hashing, `jsonwebtoken` for stateless JWT sessions.
- **Flow:** Farmer registers → password hashed (salt=10) → stored in MongoDB Atlas `users` collection → JWT issued (24h expiry) → all protected routes verify via `authenticateToken` middleware.

- **MongoDB Model:** stores name, email, phone, farm location, soil type, primary crop, irrigation type, and preferred language (`en/hi/ta/te`).

6.2 Gemini AI Chatbot (`chatbotController.js`)

The chatbot uses a **two-tier fallback architecture**:

Priority	Engine	Trigger
1st	Google Gemini Flash (`gemini-flash-latest`)	If `GEMINI_API_KEY` is set in `.env`
2nd	Built-in Agronomic Knowledge Base	If Gemini API is unreachable or key is missing

- **Multilingual Prompt Engineering:** The system prompt instructs Gemini to respond in the farmer's chosen language (Tamil/Telugu/Hindi/English) with practical farming advice formatted in bold markdown.
- **Intent Matching (Fallback):** Keyword arrays match farming topics — blight, irrigation, fertilizer, market, government schemes — and return pre-authored bilingual responses.
- **Suggested Questions:** Dynamic question chips are returned in the response payload, localised per language.

6.3 AI Crop Recommendation Engine (`cropController.js`)

A pure algorithmic **weighted scoring system** (no external ML service needed):

Parameter	Weight	Description
Nitrogen (N)	20 pts	Soil N content match to crop optimum
Phosphorus (P)	15 pts	Soil P match
Potassium (K)	15 pts	Soil K match
Temperature	18 pts	Ambient temperature agro-climate fit
Humidity	12 pts	Relative humidity compatibility
Soil pH	10 pts	pH tolerance range match
Soil Type	10 pts	Direct soil type preference bonus

- Database includes **9 major Indian crops**: Rice, Wheat, Cotton, Maize, Tomato, Sugarcane, Chickpea, Onion, Groundnut.

- Returns primary recommendation + top 2 alternatives with confidence score (45–99%).

6.4 IoT Sensor Telemetry (`iotController.js`)

- Simulates a real-time **AGRI-NODE-01** sensor node with live fluctuating values:
  - Soil Moisture (%), Soil Temperature (°C), NPK levels (kg/ha), pH, Electrical Conductivity (dS/m).
- **Auto-Mode Logic:** When soil moisture drops below 25%, the virtual pump automatically activates. Above 55%, it shuts off — mimicking a real drip controller.
- Farmer can manually override via the pump toggle button in the dashboard.

6.5 Weather Advisory Engine (`weatherController.js`)

- Calls **OpenWeatherMap API** (`/data/2.5/weather` + `/data/2.5/forecast`) when `WEATHER_API_KEY` is configured.
- Falls back to seasonal agri-meteorology model when API is unavailable.
- Returns:
  - Current conditions (temp, humidity, wind, rainfall probability, UV index, pressure).
  - 7-day forecast with `suitableForSpraying` and `suitableForHarvesting` boolean flags.
  - 3 smart advisories: Spraying window, Fungal risk alert, Irrigation scheduling.

6.6 3D Farm Visualizer (`FarmVisualizer3D.jsx`)

- Built with **pure CSS 3D transforms** and **Framer Motion** animations — no WebGL or Three.js dependency.
- Features cyber neon holographic hotspots for 4 farm zones: IoT Station, AI Drone, Micro-Drip System, Sector A Crops.
- Dual camera modes: 🌞 Daylight Drone View and 🛰️ Thermal NDVI Heatmap.
- Fully localised hotspot labels in all 4 languages.

6.7 Multilingual i18n System (`translations.js`)

- Central dictionary with keys for every UI label, tab name, microservice card description, and status message.
- Language is stored in React state in `App.jsx` and propagated to all child components as a `language` prop.
- Web Speech API (`SpeechSynthesisUtterance`) uses matching locale codes: `en-IN`, `hi-IN`, `ta-IN`, `te-IN`.

- Voice input (`webkitSpeechRecognition`) also uses `te-IN` / `ta-IN` for native script transcription.

7. Technology Stack

Backend

Technology	Version	Role
Node.js	v24.14.0	JavaScript runtime
Express.js	^4.21.2	REST API framework
Mongoose	^8.9.5	MongoDB ODM
MongoDB Atlas	Cloud	Persistent farmer database
bcryptjs	^2.4.3	Password hashing
jsonwebtoken	^9.0.2	JWT authentication
multer	^1.4.5	Image upload (disease detection)
dotenv	^16.4.7	Environment variable management

Frontend

Technology	Version	Role
React	^19.2.8	UI component framework
Vite	^8.3.0	Build tool and dev server
Tailwind CSS	^3.4.19	Utility-first styling
Framer Motion	^13.5.0	Animations and transitions
Lucide React	^1.49.0	Icon library
canvas-confetti	^1.9.4	Success celebration effects
Web Speech API	Browser	Voice input (STT) + readout (TTS)

External APIs

API	Provider	Usage
Gemini Flash	Google DeepMind	Multilingual AI agronomist chatbot
OpenWeatherMap	OpenWeather	Live weather + 7-day agri forecast

8. Database Schema

Collection: `users` (MongoDB Atlas)

```
js
{
  name:              String,         // Farmer's full name
  email:             String,         // Unique login identifier
  password:          String,         // bcrypt hashed
  phone:             String,
  location: {
    state:           String,         // e.g. "Maharashtra"
    district:        String,         // e.g. "Nashik"
    village:         String
  },
  farmDetails: {
    sizeInAcres:     Number,         // e.g. 4.5
    soilType:        String,         // "Black Soil", "Sandy Loam"...
    primaryCrop:     String,         // "Tomato", "Wheat"...
    irrigationType:  String          // "Drip Irrigation"...
  },
  preferredLanguage: enum['en','hi','mr','ta','te'],
  createdAt:         Date
}
```

> **Auto-Seeded Profile:** On first connection to Atlas, a default farmer profile (`Ramesh Patel` , `farmer@smartagri.com` , password: `farmer123`) is inserted automatically.

9. Multilingual Support Matrix

Feature	English	Hindi (हिन्दी)	Tamil (தமிழ்)	Telugu (తెలుగు)
UI Labels & Navigation	✅	✅	✅	✅
AI Chatbot Responses	✅	✅	✅	✅
Voice TTS Readout	`en-IN`	`hi-IN`	`ta-IN`	`te-IN`
Voice STT Input	`en-IN`	`hi-IN`	`ta-IN`	`te-IN`
Morning Audio Briefing	✅	✅	✅	✅
3D Holographic Hotspots	✅	✅	✅	✅
Suggested Question Chips	✅	✅	✅	✅
Fallback Agronomy Replies	✅	✅	✅	✅

10. Application Workflow (End-to-End)

11. Environment Configuration (`.env`)

```
# d:\Smartagri\server\.env
MONGODB_URI=mongodb+srv://Smartagri:<password>@smartagri.6b3xbng.mongodb.net/?
appName=Smartagri
GEMINI_API_KEY=<your_google_gemini_api_key>
WEATHER_API_KEY=<your_openweathermap_api_key>
PORT=5000
```

> **IMPORTANT**
>
> All secrets are stored exclusively in `.env` and accessed via `process.env.*` — never hardcoded in source files.

12. Running the Project

Start Backend (Express API)

```
bash
cd d:\Smartagri\server
node server.js
# Starts on http://localhost:5000
```

Start Frontend (React + Vite)

```
bash
cd d:\Smartagri\client
npm run dev -- --host
# Starts on http://localhost:5173
```

Production Build

```
bash
cd d:\Smartagri\client
npm run build
# Outputs optimised bundle to dist/ (362 kB JS, 52 kB CSS)
```

13. Key Design Decisions

Decision	Rationale
100% Pure MERN	No third-party hosting dependency (Vercel, Firebase). Full control over all 4 layers.
Two-tier AI fallback	Ensures chatbot works even without a Gemini API key configured, maintaining demo stability.
In-memory IoT simulation	Eliminates the need for physical hardware — simulates realistic sensor fluctuations and auto pump logic in pure Node.js.
Weighted scoring for crop engine	Transparent, explainable recommendation algorithm vs. a black-box ML model. Easier for report/audit.
Web Speech API (no SDK)	Zero-dependency voice I/O using native browser API with locale codes for all 4 Indian languages.
CSS 3D + Framer Motion	Zero WebGL overhead — the 3D farm visualizer runs on all devices including low-end Android phones.
MongoDB Atlas with in-memory fallback	Even if Atlas is unreachable (IP not whitelisted), the platform runs fully in demo mode without crashing.
Tailwind Dual Theme	Light Nature green theme for daytime; Cyber Neon Dark theme for night/professional use — toggled via class switching.